

A Practitioner’s Guide to Deep Learning for Individual Heterogeneity

February 2026

Abstract

This paper provides a step-by-step guide to implementing the deep learning framework for individual heterogeneity developed in [Farrell et al. \(2021\)](#) and [Farrell et al. \(2025\)](#). We show how neural networks can directly estimate heterogeneous structural parameters from individual-level data, and how influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. Our running application uses H&M fashion transaction data: we train two-tower contrastive embeddings to represent consumer preferences, estimate a multinomial choice model with heterogeneous price sensitivity, and construct influence-function-corrected confidence intervals for average elasticities and counterfactual pricing experiments. We provide complete, documented code and a companion `Python` package (`deep-inference`) that handles cross-fitting, Lambda estimation, and influence function assembly automatically. The guide is intended for applied researchers and practitioners who want to move beyond assuming homogeneous effects without sacrificing valid inference.

Keywords: deep learning, individual heterogeneity, influence functions, multinomial choice, demand estimation, contrastive learning

JEL Codes: C14, C45, D12, L81

Contents

1	Introduction	4
1.1	The Idea in Brief	4
1.2	Running Example: H&M Fashion Demand	5
1.3	What This Guide Covers	6
2	The Framework	6
2.1	Setup	6
2.2	Target Functionals	7
2.3	Why Naive Inference Fails	7
2.4	The Influence Function Correction	8
2.5	Three Regimes for Lambda	9
2.6	The Multinomial Choice Model	10
2.7	Available Models and Targets	11
3	Estimation in Practice	11
3.1	Data Requirements	11
3.2	The Algorithm	12
3.3	Cross-Fitting Details	13
3.4	Lambda Estimation: The Hessian Pipeline	14
3.5	Network Architecture	16
3.6	Diagnostics	16
3.7	Common Pitfalls	17
4	Worked Examples	18
4.1	Example 1: Linear Model (Regime B)	18
4.2	Example 2: Binary Logit (Regime C)	19
4.3	Example 3: Custom Poisson Loss	21
5	Application: H&M Fashion Demand	22
5.1	Data	22
5.2	Learning Representations	22
5.3	Mapping to the FLM Framework	22
5.4	Results	24
5.4.1	Simulation Validation	24
5.4.2	Main Inference Results	24
5.4.3	Heterogeneity in Price Sensitivity	25
5.5	Counterfactuals	25

6	Simulations	26
6.1	Coverage Validation	27
6.2	Component Diagnostics	27
6.3	Semi-Synthetic Simulations	28
6.3.1	Sequential Data with GRU Encoder	28
6.3.2	Image Data with CNN Encoder	29
6.3.3	Text Data with Transformer Embeddings	30
6.3.4	Results	30
6.4	Recommended Evaluation Workflow	31
7	Concluding Remarks	31
7.1	Other Structural Models	32
7.2	Other Targets	32
7.3	Custom Models via <code>model_from_loss()</code>	32
7.4	Computational Considerations	32
A	Implementation Details	35
A.1	Installation	35
A.2	Data Preparation	35
A.3	Inference Call	36
A.4	Running All Parameters	37
A.5	Diagnostics Interpretation Guide	38
A.6	Implementation-Theory Notes	38
A.7	Companion Repository	39

1 Introduction

Individual heterogeneity is ubiquitous in economic data. Consumers differ in their sensitivity to prices, workers respond differently to job training, and patients vary in their treatment effects. Empirical researchers have long recognized this heterogeneity and built it into their models, from discrete choice models (McFadden, 1974; Train, 2009) to personalized pricing (Dubé and Misra, 2023) and heterogeneous treatment effects more broadly. But estimating heterogeneous parameters at the individual level, and doing valid statistical inference on the resulting objects, has remained a difficult task. Traditional approaches typically assume a parametric distribution for heterogeneity (e.g., normal random coefficients) or restrict attention to a small number of discrete types. Farrell et al. (2021, 2025) develop a framework in which deep neural networks directly output individual-level structural parameters $\theta(x)$, and influence function corrections deliver valid confidence intervals despite the regularization bias inherent in neural network estimation. The discrete choice model in our application is one example; the framework applies to any structural model with a differentiable loss.

In the spirit of Nevo (2000), this paper is a practitioner’s guide to the FLM framework. We walk through the entire pipeline — from data preparation to model specification to inference to counterfactuals — using a running example that illustrates every step. We provide complete code, a companion Python package (`deep-inference`), and a simulation study validating the method. Related work on deep learning for causal inference includes DragonNet (Shi et al., 2019), which adapts network architecture for treatment effect estimation; GDR-Learners (Melnichuk and Feuerriegel, 2026), which extend doubly robust methods to heterogeneous effects; and Hetzenecker and Osterhaus (2024), who apply similar ideas to discrete choice models.

1.1 The Idea in Brief

Consider a structural model with individual-level parameters. A researcher observes outcomes Y_i , treatments or product characteristics T_i , and individual covariates X_i for $i = 1, \dots, n$ observations. The structural model specifies a loss function $\ell(Y_i, T_i, \theta)$, and the individual’s true parameters $\theta^*(X_i)$ minimize the expected loss conditional on their type X_i .

The key insight of FLM is: let a neural network directly output the structural parameters. Train a network $\hat{\theta}(X_i)$ by minimizing the structural loss across observations. Then use influence functions to correct for the regularization bias that neural networks inevitably introduce.

The framework applies broadly. For example, in a *logit* demand model, ℓ is the binary cross-entropy and $\theta(X_i) = (\alpha_i, \beta_i)$ captures individual-level intercept and price sensitivity. In a *Poisson* count model for store visits, ℓ is the Poisson negative log-likelihood and β_i is the individual’s response to a promotional campaign. In a *multinomial choice* model, ℓ is the softmax cross-entropy and $\theta(X_i)$ is a vector of taste parameters across product attributes. The target $\mu^* = E[H(X, \theta^*(X), \tilde{t})]$ can be an average parameter ($H = \theta_k$, e.g., mean price sensitivity), a marginal effect ($H = g'(\theta^* \tilde{t}) \cdot \beta$), an economic quantity like expected revenue ($H = \tilde{t} \cdot g(\theta^* \tilde{t})$), or a treatment

effect contrast ($H = g(\theta'\tilde{t}_1) - g(\theta'\tilde{t}_0)$, i.e., a difference-in-differences-style estimand).

Why is correction necessary? Neural networks use regularization (weight decay, dropout, early stopping) to prevent overfitting. This regularization shrinks estimates toward zero, making naive standard errors, computed from the Hessian of the structural loss, systematically too small. The resulting confidence intervals severely undercover. In our simulations, naive coverage can fall below 20% when the nominal level is 95%.

The influence function (IF) correction accounts for the estimation noise in $\hat{\theta}(X)$. The corrected estimator for a target functional $\mu^* = E[H(X, \theta^*(X), \tilde{t})]$ is:

$$\hat{\psi}_i = H_i - H_{\theta,i} \cdot \hat{\Lambda}_i^{-1} \cdot \ell_{\theta,i} \quad (1)$$

where H_i is the target evaluated at the estimated parameters, $H_{\theta,i}$ is the Jacobian of the target with respect to θ , $\hat{\Lambda}_i$ is the expected Hessian of the loss, and $\ell_{\theta,i}$ is the score (gradient of the loss). The average $\bar{\psi} = n^{-1} \sum_i \hat{\psi}_i$ is the debiased estimator, and its standard error provides valid inference.

The influence function has a long history in robust statistics (Hampel, 1974) and semiparametric efficiency theory (Newey and McFadden, 1994). In this setting, it serves a specific purpose: it corrects for the first-order bias introduced by estimating θ with a regularized neural network. The key property is *Neyman orthogonality*: the expectation of ψ is insensitive to first-order perturbations in $\hat{\theta}$ around θ^* . Concretely, $\partial_{\theta} E[\psi(Y, T, \theta, \Lambda)]|_{\theta^*} = 0$. This means that even though $\hat{\theta}$ converges to θ^* at a slow rate (typical for neural networks), the debiased estimator $\bar{\psi}$ achieves the parametric $\sqrt{n}$ rate. The correction term $H_{\theta}\Lambda^{-1}\ell_{\theta}$ is constructed precisely so that this orthogonality condition holds: it cancels the leading-order bias from the plug-in estimator $\bar{H}$.

1.2 Running Example: H&M Fashion Demand

Our running application uses the H&M Personalized Fashion Recommendations dataset (H&M Group, 2022), a publicly available dataset of 1.4 million fashion transactions. We estimate a multinomial choice model for online dress purchases with heterogeneous price sensitivity across consumers.

The application demonstrates the full pipeline. First, *representation learning*: we train two-tower contrastive embeddings (van den Oord et al., 2018) to learn dense representations of consumers and products from purchase histories and visual features, where consumer embeddings capture latent taste dimensions and item embeddings capture visual and categorical product attributes. Second, *choice model estimation*: consumer embeddings become the covariates X that drive heterogeneity, and the neural network maps each consumer’s 64-dimensional embedding to their structural parameters $\theta_{\text{price}}(X_i)$ and $\theta_{\text{style},k}(X_i)$. Third, *influence function inference*: the **deep-inference** package handles cross-fitting, Lambda estimation, influence function assembly, and confidence interval computation, with the entire pipeline running automatically from a five-line custom loss function. Finally, *counterfactual analysis*: we compute average price elasticities with valid confidence intervals and simulate the revenue impact of a 10% price increase, with

uncertainty bands that properly account for the neural network estimation step.

1.3 What This Guide Covers

Section 2 develops the framework: the structural model, target functionals, why naive inference fails, the IF correction, and the three Lambda estimation regimes. Section 3 covers estimation in practice: data requirements, the algorithm step by step, cross-fitting, Lambda estimation, network architecture, diagnostics, and common pitfalls. Section 4 provides three self-contained worked examples of increasing complexity (linear, logit, and custom Poisson). Section 5 presents the full H&M application with results. Section 6 validates the pipeline through simulations, including semi-synthetic studies with high-dimensional covariates. Section 7 discusses extensions and other structural models. Appendix A provides annotated code and an installation guide.

Throughout, we emphasize the *practitioner’s* perspective: what choices must be made, what can go wrong, and what to look for in the output.

2 The Framework

This section lays out the FLM framework as a practitioner would encounter it: we describe the structural model, define target functionals, explain why naive neural network inference fails, present the influence function correction, and classify problems into three “regimes” that determine how a key quantity, the expected Hessian $\Lambda(x)$, is handled.

2.1 Setup

Following Farrell et al. (2021, 2025), we observe n i.i.d. observations (Y_i, T_i, X_i) where Y_i is the outcome (chosen product, sales quantity, duration), T_i is the treatment or “action” variable (price, product attributes, dosage), and $X_i \in \mathbb{R}^{d_x}$ are individual covariates that drive heterogeneity.

A *structural model* specifies a loss function $\ell(y, t, \theta)$ parameterized by structural parameters $\theta \in \mathbb{R}^{d_\theta}$. The true parameters for individual i solve:

$$\theta^*(X_i) = \arg \min_{\theta} E[\ell(Y, T, \theta) \mid X = X_i] \quad (2)$$

Concrete example. In our H&M application, $Y_i \in \{0, 1, \dots, J-1\}$ is the chosen product from a set of J alternatives, $T_i \in \mathbb{R}^{J \times K}$ contains the attributes of each alternative (log-price, style embeddings), and $X_i \in \mathbb{R}^{64}$ is the consumer’s learned embedding. The structural parameters $\theta^*(X_i) = (\beta_{\text{price}}(X_i), \beta_{\text{style},1}(X_i), \dots, \beta_{\text{style},5}(X_i))$ are the consumer’s taste parameters, and the loss is the multinomial logit negative log-likelihood:

$$\ell(y, t, \theta) = -V_y + \log \sum_{j=0}^{J-1} e^{V_j}, \quad V_j = x'_j \theta \quad (3)$$

The framework is *not* limited to discrete choice. Table 1 lists the structural models currently available in the **deep-inference** package, along with their loss functions and parameter dimensions.

Table 1: Available structural models in the **deep-inference** package.

Model	Loss $\ell(y, t, \theta)$	Link	d_θ
Linear	$(y - \alpha - \beta t)^2$	Identity	2
Logit	$\log(1 + e^\eta) - y\eta$	Logistic	2
Poisson	$e^\eta - y\eta$	Log	2
Gamma	$y/\mu + \log \mu$	Log	2
Multinomial	$-V_y + \log \sum_j e^{V_j}$	Softmax	$(J-1)+K$
Custom	Any differentiable $\ell(y, t, \theta)$	Any	User-specified

Note: $\eta = \alpha + \beta t$, $\mu = g^{-1}(\eta)$. The package includes over a dozen families (Weibull, Gumbel, Tobit, NegBin, Probit, Beta, ZIP, and others). Custom losses can be supplied via the `loss=` argument.

2.2 Target Functionals

We are usually not interested in $\theta^*(X_i)$ for every individual, but rather in some summary: an *average* of a function of θ^* across the population:

$$\mu^* = E[H(X, \theta^*(X), \tilde{t})] \quad (4)$$

where H is the *target function* and $\tilde{t}$ is an evaluation point. Common targets include:

Table 2: Target functionals for inference.

Target	$H(x, \theta, \tilde{t})$	Interpretation
Average parameter	θ_k	$E[\beta_{\text{price}}]$: mean price sensitivity
Average marginal effect	$g'(\alpha + \beta \tilde{t}) \cdot \beta$	Mean marginal effect at $\tilde{t}$
Elasticity	$\beta \cdot \tilde{t} \cdot (1 - p)$	Price elasticity of demand
Dose-response	$g(\alpha + \beta \tilde{t})$	Predicted outcome at $\tilde{t}$
Profit	$\tilde{t} \cdot g(\alpha + \beta \tilde{t})$	Expected revenue
DID / ATE	$g(\theta' \tilde{t}_1) - g(\theta' \tilde{t}_0)$	Treatment effect contrast
Custom	Any $H(x, \theta, \tilde{t})$	User-specified

The Jacobian $H_\theta = \partial H / \partial \theta$ is required for the IF correction. For built-in targets, closed-form Jacobians are provided. For custom targets, the package computes them via automatic differentiation.

2.3 Why Naive Inference Fails

To estimate μ^* , we train a neural network $\hat{\theta}(X)$ by minimizing $\sum_i \ell(Y_i, T_i, \hat{\theta}(X_i))$ with sample splitting and then compute $\hat{\mu} = n^{-1} \sum_i H(X_i, \hat{\theta}(X_i), \tilde{t})$.

The *naive* standard error treats $\hat{\theta}$ as if it were the truth and computes $\text{SE}_{\text{naive}} = \text{sd}(H_i)/\sqrt{n}$. This is wrong because $\hat{\theta}$ is itself estimated with error, and neural network regularization introduces systematic bias.

The problem is quantitatively severe. In our simulations (Section 6), naive 95% CIs achieve only 0–20% actual coverage, the naive SE is 3–10 $\times$ too small, and the bias is toward zero because regularization shrinks $\hat{\theta}$.

This is not a small-sample problem; it persists at $n = 50,000$. The regularization bias is a *feature* of neural network training (it prevents overfitting) but becomes a *bug* for inference.

In code, the difference is stark:

```

1 # Naive SE (wrong): ignores estimation uncertainty
2 se_naive = np.std(H_values) / np.sqrt(n)
3 # IF SE (correct): accounts for neural network estimation
4 se_if = result.se # from influence function
5 print(f"Naive: {se_naive:.4f}, IF: {se_if:.4f}, Ratio: {se_if/se_naive:.1f}x")
6 # Typical output for logit: Naive=0.009, IF=0.026, Ratio=2.8x

```

To see why formally, consider the decomposition of the naive estimator’s error:

$$\sqrt{n}(\hat{\mu}_{\text{naive}} - \mu^*) = \underbrace{\frac{1}{\sqrt{n}} \sum_{i=1}^n [H_i - \mu^*]}_{\text{sampling noise}} + \underbrace{\frac{1}{\sqrt{n}} \sum_{i=1}^n H_{\theta,i} (\hat{\theta}_i - \theta_i^*)}_{\text{regularization bias}} + o_P(1) \quad (5)$$

The first term is standard sampling variability and vanishes at rate $\sqrt{n}$. The second term captures the bias introduced by the neural network’s regularization: because $\hat{\theta}$ is systematically shrunk toward zero (by weight decay, dropout, and early stopping), $\hat{\theta}_i - \theta_i^*$ is not mean-zero and this term does not vanish. The IF correction in the next section is designed precisely to cancel this second term.

2.4 The Influence Function Correction

The influence function correction of [Farrell et al. \(2025\)](#) (Theorem 2) removes the regularization bias. For each observation i , define:

$$\psi_i = H_i - H_{\theta,i} \cdot \Lambda_i^{-1} \cdot \ell_{\theta,i} \quad (6)$$

where $H_i = H(X_i, \hat{\theta}(X_i), \tilde{t})$ is the target evaluated at the estimated parameters, $H_{\theta,i} = \partial H / \partial \theta|_{\hat{\theta}(X_i)} \in \mathbb{R}^{1 \times d_\theta}$ is the target Jacobian, $\Lambda_i = E[\nabla_\theta^2 \ell(Y, T, \theta) \mid X = X_i]|_{\hat{\theta}(X_i)} \in \mathbb{R}^{d_\theta \times d_\theta}$ is the expected Hessian of the loss conditional on covariates, and $\ell_{\theta,i} = \nabla_\theta \ell(Y_i, T_i, \hat{\theta}(X_i)) \in \mathbb{R}^{d_\theta}$ is the score (gradient of the loss at the observed data point).

The debiased estimator and its standard error are:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n \psi_i, \quad \widehat{\text{SE}} = \frac{\text{sd}(\psi_1, \dots, \psi_n)}{\sqrt{n}} \quad (7)$$

Intuition. The correction term $H_\theta \Lambda^{-1} \ell_\theta$ is an “adjustment for what the neural net got wrong.” The score ℓ_θ measures how far observation i ’s data is from the fitted model. The Hessian Λ converts this into parameter-space error. The Jacobian H_θ maps this to the target. When the neural net fits perfectly (scores near zero), the correction vanishes. When it makes systematic errors (regularization bias), the correction kicks in.

Neyman orthogonality. The IF ψ in Equation (6) satisfies the *Neyman orthogonality* condition:

$$\left. \frac{\partial}{\partial \theta} E[\psi(Y, T, \theta, \Lambda)] \right|_{\theta=\theta^*, \Lambda=\Lambda^*} = 0 \quad (8)$$

The correction term $H_\theta \Lambda^{-1} \ell_\theta$ is constructed precisely so that first-order perturbations in $\hat{\theta}$ around θ^* cancel. This is why the debiased estimator $\hat{\mu} = \bar{\psi}$ achieves $\sqrt{n}$ -consistency despite the neural network’s slow convergence rate: the estimation error in $\hat{\theta}$ enters only through a second-order remainder, which is negligible at the $\sqrt{n}$ scale.

Connection to classical statistics. Equation (6) is a semiparametric influence function in the tradition of [Hampel \(1974\)](#) and [Newey and McFadden \(1994\)](#). The specific form arises from the “one-step correction” or “debiasing” approach used widely in semiparametric inference ([Chernozhukov et al., 2018](#)). The Neyman orthogonality property is central to the broader program of orthogonal statistical learning ([Foster and Syrgkanis, 2023](#)); related automatic debiasing approaches include RieszNet ([Chernozhukov et al., 2022](#)). The contribution of [Farrell et al. \(2021, 2025\)](#) is showing that this correction remains valid when the first-stage nuisance (the structural parameters) is estimated by a deep neural network.

2.5 Three Regimes for Lambda

The expected Hessian $\Lambda(X) = E[\nabla_\theta^2 \ell(Y, T, \theta) \mid X]$ depends on how the model and data are structured. [Farrell et al. \(2025\)](#) identify three cases:

Table 3: Three Lambda estimation regimes.

Regime	When	Lambda method	Cross-fitting
A	RCT, known F_T	Compute (MC integration)	2-way
B	Linear model	Analytic (closed-form)	2-way
C	Obs. + nonlinear	Estimate (ridge)	3-way

Regime A: Randomized experiment with known treatment distribution. If T is randomly assigned with known distribution F_T , and the Hessian does not depend on Y , then $\Lambda(X)$ can be computed via Monte Carlo integration: $\Lambda(X) = \int \nabla_{\theta}^2 \ell(y, t, \theta) dF_T(t)$. This is the simplest case and requires only two-way cross-fitting. Typical examples include A/B tests where treatment assignment probabilities are known by design, and randomized pricing experiments where the price distribution is controlled by the researcher. The `inference()` API accepts `is_randomized=True` and a `treatment_dist` argument to trigger this regime.

Regime B: Linear model. For linear models, the Hessian is constant ($\nabla_{\theta}^2 \ell = 2$), so $\Lambda(X) = 2E[TT' | X]$, which can be estimated analytically from the data without a separate estimation step. This avoids the need for three-way splitting entirely, making the method fast and reliable even at moderate sample sizes. The package detects Regime B automatically when `model='linear'` is specified.

In the `inference()` API, the regime is selected automatically based on the model and data:

```

1  from deep_inference import inference
2  from deep_inference.lambda_.compute import Normal
3
4  # Regime A: randomized experiment with known F_T
5  result = inference(Y, T, X, model='logit', target='beta',
6                    is_randomized=True, treatment_dist=Normal(0, 1))
7
8  # Regime B: linear model (auto-detected)
9  result = inference(Y, T, X, model='linear', target='beta')
10
11 # Regime C: observational + nonlinear (auto-detected)
12 result = inference(Y, T, X, model='logit', target='beta')
```

Regime C: Observational data with nonlinear model. In most applied settings, including our H&M application, the Hessian depends on θ (e.g., through $p(1-p)$ in logit or softmax probabilities in multinomial logit). Since θ is estimated, we must *estimate* Λ via ridge regression, which requires a three-way sample split. Section 3 discusses the implementation details; the package handles the splitting automatically.

2.6 The Multinomial Choice Model

Our application uses a multinomial logit (conditional logit) model (McFadden, 1974; Berry et al., 1995; Train, 2009). Consumer i chooses from J alternatives, each described by K attributes $x_{ij} \in \mathbb{R}^K$. Utilities are:

$$V_{ij} = x'_{ij} \theta(X_i), \quad P(Y_i = j | X_i) = \frac{e^{V_{ij}}}{\sum_{m=1}^J e^{V_{im}}} \quad (9)$$

The FLM framework estimates $\theta(X_i)$ for each individual using individual-level data and neural networks, rather than assuming a parametric distribution for heterogeneity; see also [Hetzenecker and Osterhaus \(2024\)](#) for a related approach to heterogeneous discrete choice.

The connection between two-tower embeddings and the FLM framework is natural: the consumer embedding X_i captures the latent heterogeneity dimensions, and the neural network maps $X_i \rightarrow \theta(X_i)$. The item attributes T_i include prices and (PCA-reduced) item embeddings.

2.7 Available Models and Targets

The `deep-inference` package provides a growing library of built-in model families (Table 1) and target functionals (Table 2). For models not in the built-in library, users can supply a custom loss function:

```

1  import torch
2  from deep_inference import inference
3
4  def my_loss(y, t, theta):
5      """Custom structural loss."""
6      mu = torch.exp(theta[0] + theta[1] * t) # log-link
7      return mu - y * torch.log(mu)          # Poisson NLL
8
9  result = inference(Y, T, X, loss=my_loss, theta_dim=2,
10                    target_fn=lambda x, th, tt: th[1])

```

The package automatically determines whether the Hessian depends on θ (triggering Regime C and three-way splitting) and computes all required derivatives via automatic differentiation. The user needs only the loss function and the target.

3 Estimation in Practice

This section walks through every step of the estimation procedure, discusses the choices practitioners must make, and flags the pitfalls that can silently invalidate results. For each step, we present the relevant equation from [Farrell et al. \(2025\)](#) alongside the corresponding implementation in the `deep-inference` package, making explicit how theory maps to code.

3.1 Data Requirements

The method requires individual-level data in triplets (Y_i, T_i, X_i) . Outcomes Y may be continuous (sales, duration), binary (purchase, click), count (visits), or discrete choice (chosen alternative index). Treatments or actions T are the variables whose effect varies across individuals: for discrete choice, T contains the attributes of all alternatives (prices, features); for treatment effects, T is the treatment indicator or dose. Covariates X are the individual characteristics that drive heterogeneity

and serve as the neural network’s inputs; higher-dimensional X allows the network to capture richer heterogeneity patterns but requires more data.

Sample size considerations. Neural networks are data-hungry. As a rough guide, $n \geq 2,000$ suffices for binary logit with $d_\theta = 2$, while multinomial logit with $d_\theta \leq 6$ requires $n \geq 5,000$, and models using three-way splitting (Regime C) need $n \geq 8,000$ because the split reduces effective training data to about 60% of n .

Choice set construction. For multinomial choice models, each observation requires a full choice set: the chosen item plus $J - 1$ alternatives. When the full choice set is too large (e.g., 100,000 products), sample $J - 1$ random alternatives per occasion. This is consistent with McFadden’s sampling theorem and standard in the IO literature (Train, 2009). We use $J = 20$ in our application.

3.2 The Algorithm

The estimation proceeds in five steps, all handled internally by the `inference()` function:

Step 0: Prepare data. Encode T as a numeric array with consistent shape across observations. For multinomial logit, pack alternative attributes as $(n, J \times K)$.

Step 1: Cross-fitting. Split data into K folds. For each fold k , train the structural network on the remaining $K - 1$ folds:

$$\hat{\theta}_k(\cdot) = \arg \min_{\theta \in \mathcal{F}} \sum_{i \in I_k^{(\theta)}} \ell(Y_i, T_i, \theta(X_i)) \quad (10)$$

where $\mathcal{F}$ is the neural network function class and $I_k^{(\theta)}$ denotes the theta-training subset of fold k ’s non-evaluation data.¹ Then predict $\hat{\theta}_k(X_i)$ on the held-out fold I_k . This produces out-of-sample estimates $\hat{\theta}(X_i)$ for all i .

Step 2: Estimate Lambda. Compute individual Hessians via automatic differentiation:

$$H_i = \nabla_\theta^2 \ell(Y_i, T_i, \hat{\theta}(X_i)) \in \mathbb{R}^{d_\theta \times d_\theta} \quad (11)$$

Then estimate the conditional expectation $\Lambda(x) = E[H_i \mid X = x]$ by regressing H_i on X_i :

$$\hat{\Lambda}(x) = \text{vec}_{\text{sym}}^{-1}(\hat{\beta}_0 + \hat{\beta}_1 x), \quad \hat{\beta} = \arg \min_{\beta} \sum_{i \in I_k^{(\lambda)}} \|\text{vech}(H_i) - \beta_0 - \beta_1 X_i\|^2 + \alpha \|\beta_1\|^2 \quad (12)$$

where $\text{vech}(\cdot)$ extracts the $d_\theta(d_\theta + 1)/2$ unique elements of the symmetric Hessian, $I_k^{(\lambda)}$ is the Lambda-training subset, and $\alpha = 1000$ is the ridge penalty. Section 3.4 discusses this step in

¹In the implementation (`engine/crossfit.py`, lines 244–351), `run_crossfit()` orchestrates this loop: for each fold, it calls `train_structural_net()` on the training indices, then predicts $\hat{\theta}(X_i)$ on the held-out fold.

detail.

Step 3: Compute scores and Jacobians. Evaluate the score $\ell_\theta(Y_i, T_i, \hat{\theta}(X_i))$ and target Jacobian $H_\theta(X_i, \hat{\theta}(X_i), \tilde{t})$ at each observation. For built-in targets (e.g., average parameter), the Jacobian has a known closed form. For custom targets, it is computed via reverse-mode automatic differentiation using `torch.func.jacrev` inside `torch.vmap` for efficient batched evaluation.²

Step 4: Assemble ψ and compute SE. Combine the pieces into the influence function and derive the final estimate:

$$\psi_i = H_i - H_{\theta,i} \hat{\Lambda}_i^{-1} \ell_{\theta,i}, \quad \hat{\mu} = \frac{1}{n} \sum_{i=1}^n \psi_i, \quad \widehat{\text{SE}} = \sqrt{\frac{1}{n(n-1)} \sum_{i=1}^n (\psi_i - \hat{\mu})^2} \quad (13)$$

The Bessel correction $(n-1)$ in the variance ensures unbiased variance estimation. Confidence intervals follow from asymptotic normality: $\hat{\mu} \pm z_{\alpha/2} \cdot \widehat{\text{SE}}$.³

In code, this entire pipeline is a single function call:

```

1  from deep_inference import inference
2
3  result = inference(Y, T, X, loss=my_loss, theta_dim=6,
4                    target_fn=my_target,
5                    hessian_depends_on_theta=True,
6                    n_folds=50, epochs=300, patience=50)
7
8  print(f"Estimate: {result.mu_hat:.4f} +/- {result.se:.4f}")
9  print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")

```

3.3 Cross-Fitting Details

Cross-fitting is essential for valid inference. It prevents the neural network from “seeing” the same data used to evaluate the influence function, eliminating a bias term that would otherwise require undersmoothing conditions.

Number of folds. We recommend $K = 50$ folds. With fewer folds, each training set is smaller, potentially degrading $\hat{\theta}$. With more folds, computational cost increases linearly. In our experience, $K = 50$ provides a good balance for $n \geq 2,000$. At smaller sample sizes, $K = 20$ may be necessary.

²See `autodiff/jacobian.py`, lines 42–82. The `vmap` batching computes all n Jacobians in a single vectorized call, avoiding a Python loop.

³Implementation in `engine/assembler.py`, lines 56–85: the einsum `torch.einsum('nd,nde->ne', h_jacobian, lambda_inv)` computes $H_\theta \Lambda^{-1}$, then the dot product with the score yields the correction term. Variance computation is in `engine/variance.py`, lines 125–149.

Two-way vs. three-way splitting. When the Hessian depends on θ (Regime C), estimating Λ on the same data used to estimate θ would introduce dependence. The three-way split resolves this: within each fold, the $K - 1$ non-evaluation folds are further split so that $\hat{\theta}$ and $\hat{\Lambda}$ are estimated on independent subsets. Specifically, 60% of the training data goes to $I_k^{(\theta)}$ and 40% to $I_k^{(\lambda)}$.⁴ The effective training sample for $\hat{\theta}$ drops to about 60% of n ; see Section 6.1 for the resulting coverage impact.

Practical implication. Three-way splitting is triggered automatically when the package detects that the Hessian depends on θ . For the multinomial logit, this is always the case (softmax probabilities depend on θ). The practitioner does not need to manage the splitting manually, but should be aware that more data is needed than for linear models.

3.4 Lambda Estimation: The Hessian Pipeline

Lambda estimation is the most delicate step in the pipeline. An incorrect $\hat{\Lambda}$ directly corrupts the standard errors. This section describes the full pipeline from per-observation Hessians to the regularized, positive-semidefinite $\hat{\Lambda}(X)$ that enters the IF formula.

Step 1: Per-observation Hessians. For each observation in $I_k^{(\lambda)}$, the package computes the $d_\theta \times d_\theta$ Hessian of the loss with respect to θ , evaluated at the cross-fitted $\hat{\theta}(X_i)$. The implementation first checks whether the structural model provides a closed-form `hessian()` method; if not, it falls back to automatic differentiation via nested reverse-mode calls: `torch.func.jacrev(jacrev(loss))` applied per observation through `vmap`.⁵

Step 2: Symmetry exploitation. Each Hessian H_i is symmetric, so only the $d_\theta(d_\theta + 1)/2$ unique upper-triangle elements need to be stored and regressed. The implementation extracts these via `hessians[:, triu_idx[0], triu_idx[1]]`, reducing memory and the number of regression targets. After prediction, the full symmetric matrix is reconstructed.

Step 3: Ridge regression. Each unique element of the upper triangle is treated as a separate regression target, and a single ridge regression (Hoerl and Kennard, 1970) fits all targets simultaneously:

$$\hat{\beta} = \arg \min_{\beta} \sum_{i \in I_k^{(\lambda)}} \|\text{vech}(H_i) - X_i' \beta\|^2 + \alpha \|\beta\|^2, \quad \alpha = 1000 \quad (14)$$

The strong regularization ($\alpha = 1000$) biases $\hat{\Lambda}(X)$ toward the population mean Hessian. This sacrifices accuracy in capturing heterogeneity in Λ (correlation with oracle: 0.508) but produces reliable standard errors (96% coverage). The high α may seem counterintuitive, but recall that Λ

⁴The split fraction is configurable via `three_way_theta_frac` in `engine/crossfit.py`, line 155. The 60/40 default reflects that $\hat{\theta}$ (a neural network) is more data-hungry than $\hat{\Lambda}$ (a ridge regression).

⁵See `lambda/_estimate.py`, lines 87–154. Closed-form Hessians are available for the built-in logit ($p(1 - p)$), Poisson (λ), and linear (constant) models. Custom losses always use autodiff.

is not the object of interest; it is a nuisance quantity in the IF formula. The Neyman orthogonality property established in [Farrell et al. \(2025\)](#) ensures that first-order errors in $\hat{\Lambda}$ do not corrupt $\hat{\mu}$. We only need $\hat{\Lambda}$ “close enough” for the second-order remainder to be negligible.

Step 4: PSD projection. Ridge predictions can produce non-positive-semidefinite matrices. Since Λ^{-1} appears in the IF formula, we need $\hat{\Lambda}(X_i) \succ 0$. The package performs an eigendecomposition of each predicted $\hat{\Lambda}(X_i)$ and clamps eigenvalues:

$$\hat{\Lambda}(X_i) = VD_+V', \quad (D_+)_{jj} = \max\left(D_{jj}, \frac{\lambda_{\max}}{c_{\max}}\right) \quad (15)$$

where V, D are from the eigendecomposition, $\lambda_{\max}$ is the largest eigenvalue, and $c_{\max} = 100$ is the maximum allowed condition number.⁶

Step 5: Regularized inversion. Even after PSD projection, a small ridge (10^{-4}) is added to the diagonal before inversion: $\hat{\Lambda}_i^{-1} = (\hat{\Lambda}_i + 10^{-4}I)^{-1}$. This is a separate safeguard from the PSD projection, preventing numerical blow-up when eigenvalues are small but positive.⁷

Checking Lambda diagnostics. After running inference, the key Lambda diagnostics can be inspected:

```
1 # After inference: check Lambda diagnostics
2 result = inference(Y, T, X, loss=my_loss, theta_dim=2, ...)
3
4 print(f"Lambda min eigenvalue: {result.diagnostics['min_lambda_eig']:.6f}")
5 print(f"Correction ratio: {result.diagnostics['correction_ratio']:.1f}")
6 # In simulation: compare IF SE to empirical SE
7 # print(f"SE ratio: {result.se / empirical_se:.3f}")
```

A minimum eigenvalue above 10^{-4} and a correction ratio consistent with the model family (2–3 for binary logit, 70–90 for multinomial logit) indicate a healthy Lambda estimation.

A note on MLP estimation. An MLP Lambda estimator achieves the highest correlation with the oracle (0.997) yet produces substantially undercovering standard errors (67% coverage). The MLP overfits the individual Hessians, producing $\hat{\Lambda}(X)$ estimates that are too variable. Since Λ^{-1} appears in the IF formula, extreme Lambda values create extreme ψ_i values, inflating the variance estimator. In practice, coverage tends to be a more informative metric than correlation for evaluating Lambda estimators.

⁶See `lambda/_estimate.py`, lines 324–367 (`_project_to_psd`). The condition number bound prevents extreme eigenvalue ratios that would amplify noise through Λ^{-1} .

⁷Minimum Lambda eigenvalue $> 10^{-4}$ is the diagnostic threshold in the validation suite (`evals/common/metrics.py`). Values below this trigger a warning.

Table 4: Lambda estimation methods: comparison on logit simulation ($n = 5,000$, $M = 50$, eval_06).

Method	Corr. w/ Oracle	Coverage	SE Ratio	Notes
Ridge ($\alpha = 1000$)	0.508	96%	0.91	Default, recommended
Aggregate	0.000	95%	1.00	Ignores X -dependence
LGBM	0.978	96%	1.00	Good accuracy
MLP	0.997	67%	0.65	Not recommended

3.5 Network Architecture

The structural network maps $X \rightarrow \theta(X)$ via a multi-layer perceptron (Hornik et al., 1989). The default architecture uses two hidden layers with decreasing width ([64, 32]), which rarely needs tuning; larger networks ([128, 64, 32]) may help with very rich heterogeneity patterns but require more data. The learning rate is 0.01, standard for the Adam optimizer with structural losses. Training runs for 200 epochs on binary models and 300 on multinomial models, with a fixed dropout rate of 0.1 for mild regularization.

The early stopping patience parameter deserves special emphasis. We recommend at least 50 for multinomial models. With three-way splitting, the validation set is small and noisy, causing premature stopping if patience is too low. With `patience=10` (a common default), the network stops at epoch 15–20 and severely underfits; with `patience=50`, convergence occurs at epoch 150+, producing valid coverage. This is the single most common pitfall we have encountered.

3.6 Diagnostics

After running `inference()`, several diagnostic quantities should be checked.

Minimum Lambda eigenvalue. The smallest eigenvalue of $\hat{\Lambda}$ should exceed 10^{-4} . If the smallest eigenvalue is too close to zero, the Hessian is nearly singular, which typically means the model is poorly identified or the sample size is too small. The package adds ridge regularization (10^{-4}) to prevent numerical blow-up in the matrix inversion, but prints a warning when eigenvalues are suspiciously small. If this warning appears, consider reducing d_θ (e.g., via stronger PCA compression of attributes), increasing n , or switching to a different Lambda estimation method.

Correction ratio. The ratio $\text{Var}(\text{correction})/\text{Var}(H_i)$ measures how much the IF correction contributes relative to the plug-in variability. For binary logit, values of 2–3 are normal. For multinomial logit, values of 70–90 are normal due to the higher-dimensional θ .⁸ Large correction ratios for multinomial models are expected and should not be cause for concern; they simply reflect the fact that the IF correction is doing more work to account for the estimation uncertainty across a larger parameter vector.

⁸Correction ratios of 70–90 for multinomial logit confirmed across $M = 50$ replications in eval_09. For binary logit, ratios of 2–3 are typical (eval_06).

SE ratio. In simulation studies where the truth is known, the ratio of estimated SE to empirical SE should lie in $[0.7, 1.5]$ and ideally be close to 1. Values below 0.7 suggest that the estimated standard errors understate uncertainty, while values above 1.5 suggest overestimation. On real data, this diagnostic is not directly available, but the other checks (eigenvalues, correction ratio, and simulation validation on a matched DGP) provide confidence that the standard errors are well-calibrated.

z -score distribution. If running multiple targets or a simulation study, the z -scores $(\hat{\mu} - \mu^*)/\text{SE}$ should have mean ≈ 0 and standard deviation ≈ 1 . A mean significantly below zero indicates that the estimator is systematically biased downward (common when patience is too low), while a standard deviation much larger than 1 suggests that the standard errors are too small. Table 10 in Section 6 lists the centralized pass/fail thresholds for all diagnostics.

3.7 Common Pitfalls

Patience too low. In our experience, low patience is the single most common source of poor results. With three-way splitting, the validation set is small (about 7% of data with $K = 50$), making the validation loss noisy. A patience of 10 (a common default in deep learning) triggers early stopping at epoch 15–20, well before the network has learned the structural relationship. With `patience=50`, convergence typically occurs at epoch 150+, producing valid coverage. We recommend at least 50 for all nonlinear models using three-way splitting.

MLP Lambda. Using an MLP to estimate $\Lambda(X)$ produces the highest correlation with the oracle (0.997) but invalid standard errors (67% coverage instead of 95%). The MLP overfits the individual Hessians, producing $\hat{\Lambda}(X)$ estimates that are too variable; since Λ^{-1} appears in the IF formula, this variability corrupts the influence function values. The default ridge estimator ($\alpha = 1000$) sacrifices correlation but delivers valid coverage (Section 3.4).

Sample size too small. Three-way splitting with $K = 50$ folds means each fold has $n/50$ observations, of which 60% are used for $\hat{\theta}$ training. At $n = 2,000$, that is only 24 observations per fold for training, which is borderline for a neural network with $d_\theta = 6$. Our simulation results confirm that coverage degrades at small n : for multinomial logit, coverage is 88% at $n = 5,000$ but rises to 98% at $n = 8,000$. As a rule of thumb, use $n \geq 5,000$ for binary models and $n \geq 8,000$ when three-way splitting is active.

Ignoring diagnostics. The package prints warnings for suspicious Lambda eigenvalues and correction ratios. These warnings should not be dismissed: a minimum eigenvalue below 10^{-4} may indicate poor model identification, and a correction ratio far outside the expected range for the model family (Section 3.6) suggests Lambda estimation problems that will propagate into the standard errors.

Non-differentiable loss. The IF correction requires computing the Hessian $\nabla_{\theta}^2 \ell$ via automatic differentiation. If the loss function contains non-differentiable operations (`int()`, `if/else` on tensor values, floor division, or other discrete operations), the autodiff computation will fail or produce incorrect gradients. All operations in the loss function must use `torch` tensor operations that support gradient tracking.

Data-dependent indexing. For multinomial logit with reordered choice sets (chosen alternative always at index 0), use `V[0]` instead of `V[int(y)]`. The `int()` call breaks `torch.vmap`, which is used internally for batched Hessian computation.⁹ The solution is to reorder the choice set during data preparation so that the chosen alternative is always at position 0, then use the constant index `V[0]` in the loss.

For readers who want to try the method before diving into the full application, Section 4 provides three progressively richer worked examples: linear (Regime B), binary logit (Regime C), and custom Poisson loss.

4 Worked Examples

Before presenting the full H&M application, we walk through three progressively richer examples that illustrate the key features of the framework. Each example is self-contained and can be run directly.

4.1 Example 1: Linear Model (Regime B)

The simplest case is a linear model with heterogeneous treatment effects. Here the Hessian is constant, so Lambda has a closed form (Regime B), only two-way splitting is needed, and the method is fast.

```

1  import numpy as np
2  from deep_inference import inference
3
4  # DGP: Y = alpha(X) + beta(X) * T + eps
5  np.random.seed(42)
6  n = 3000
7  X = np.random.randn(n, 5)
8  T = np.random.randn(n)
9  alpha_true = 1.0 + 0.3 * X[:, 0]
10 beta_true = 0.5 + 0.2 * X[:, 1]      # Heterogeneous effect
11 Y = alpha_true + beta_true * T + np.random.randn(n)
12
13 # Run inference: target is E[beta] = 0.5

```

⁹This is a subtle issue: `vmap` requires static indexing because it traces a single computation graph for all batch elements. Dynamic indexing via `int(y)` produces a different graph per observation, which `vmap` cannot handle.

```

14 result = inference(Y, T, X, model='linear', target='beta')
15 print(f"E[beta] = {result.mu_hat:.3f} +/- {result.se:.3f}")
16 print(f"95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")
17 # Expected: E[beta] ~ 0.5, Regime B (analytic Lambda, 2-way split)

```

Because the Hessian is constant ($\nabla_{\theta}^2 \ell = 2$ for the squared loss), the package detects Regime B automatically: Lambda is computed in closed form, only two-way cross-fitting is used, and the entire pipeline finishes in under a minute. The IF correction here adjusts for the regularization bias in $\hat{\beta}(X)$, and the resulting confidence interval covers the true $E[\beta] = 0.5$ at the nominal rate.

Monte Carlo validation. To verify coverage, we repeat this example $M = 50$ times with fresh data seeds. At each replication, we record whether the 95% CI covers the true $\mu^* = 0.5$:

```

1 # MC validation loop (M=50 replications)
2 coverages = []
3 for seed in range(1, 51):
4     np.random.seed(seed)
5     X = np.random.randn(3000, 5)
6     T = np.random.randn(3000)
7     Y = (1.0 + 0.3*X[:,0]) + (0.5 + 0.2*X[:,1])*T + np.random.randn
          (3000)
8     r = inference(Y, T, X, model='linear', target='beta')
9     coverages.append(r.ci_lower <= 0.5 <= r.ci_upper)
10 print(f"Coverage: {np.mean(coverages):.0%}") # Expected: ~94-96%

```

Table 5 reports the results. All diagnostics pass, confirming that the linear model with Regime B delivers valid inference even at modest sample sizes.

Table 5: Linear model Monte Carlo validation ($M = 50$, $n = 3,000$, Regime B).

Metric	Value	Threshold	Status
Coverage (95% CI)	94%	[90%, 99%]	PASS
SE ratio	1.02	[0.7, 1.5]	PASS
Bias	0.003	< 0.05	PASS
z-mean	0.05	(-0.3, 0.3)	PASS

Note: Results from `evals/eval_linear_logit_examples.py`. Regime B uses two-way splitting with analytic Lambda; no Lambda estimation errors can occur.

4.2 Example 2: Binary Logit (Regime C)

The binary logit introduces the key complication: the Hessian depends on θ through $p(1 - p)$, so the package switches to Regime C with three-way splitting. This example also demonstrates diagnostics.

```

1 import numpy as np
2 from deep_inference import inference
3
4 # DGP: logit model with heterogeneous beta
5 np.random.seed(42)
6 n = 5000
7 X = np.random.randn(n, 5)
8 T = np.random.randn(n)
9 beta_true = 0.5 + 0.3 * X[:, 0]
10 p = 1 / (1 + np.exp(-(1.0 + beta_true * T)))
11 Y = np.random.binomial(1, p).astype(float)
12
13 # Run inference
14 result = inference(Y, T, X, model='logit', target='beta',
15                   n_folds=20, epochs=200, patience=50)
16 print(f"E[beta] = {result.mu_hat:.3f} +/- {result.se:.3f}")
17 print(f"95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")
18
19 # Check diagnostics
20 diag = result.diagnostics
21 print(f"Lambda min eigenvalue: {diag['min_lambda_eig']:.6f}")
22 print(f"Correction ratio: {diag['correction_ratio']:.1f}")

```

Two things to notice. First, the IF standard error is larger than the naive SE (typically by a factor of 2–3× for binary logit), reflecting the additional uncertainty from neural network estimation. Second, the correction ratio of 2–3 is normal for binary logit and indicates that the IF correction is contributing meaningfully.

Monte Carlo validation. Running $M = 50$ replications at $n = 5,000$ shows the contrast between naive and IF inference:

Table 6: Binary logit Monte Carlo validation ($M = 50$, $n = 5,000$, Regime C).

Method	Coverage	SE Ratio	Bias	z-mean
IF (corrected)	96%	0.91	0.006	−0.06
Naive	18%	—	0.006	—

Note: Results from `evals/eval_linear_logit_examples.py`. IF SE is 2.5–3× larger than naive SE.

The naive method achieves only 18% coverage — its confidence intervals are roughly half as wide as they should be. The IF correction restores valid coverage by accounting for the estimation uncertainty in $\hat{\theta}$. This is the central result: for nonlinear models, the IF correction is not optional. The following code computes both SEs for comparison:

```

1 # Naive vs IF SE comparison

```

```

2  beta_hats = result.theta_hat[:, 1] # Individual beta estimates
3  se_naive = np.std(beta_hats) / np.sqrt(n)
4  se_if = result.se
5
6  print(f"Naive SE: {se_naive:.4f}")
7  print(f"IF SE:    {se_if:.4f}")
8  print(f"Ratio:    {se_if / se_naive:.1f}x")
9  # Typical output: Naive SE=0.009, IF SE=0.026, Ratio=2.8x

```

4.3 Example 3: Custom Poisson Loss

The custom loss interface allows any differentiable model to be used without writing new model code. Here we define a Poisson regression in three lines and run the full IF pipeline.

```

1  import torch
2  import numpy as np
3  from deep_inference import inference
4
5  # DGP: Poisson with log-link
6  np.random.seed(42)
7  n = 5000
8  X = np.random.randn(n, 5)
9  T = np.random.randn(n)
10 alpha_true = 0.5 + 0.2 * X[:, 0]
11 beta_true = -0.3 + 0.1 * X[:, 1]
12 lam = np.exp(alpha_true + beta_true * T)
13 Y = np.random.poisson(lam).astype(float)
14
15 # Custom Poisson loss (3 lines)
16 def poisson_loss(y, t, theta):
17     mu = torch.exp(theta[0] + theta[1] * t)
18     return mu - y * torch.log(mu)
19
20 # Run inference: E[beta] = -0.3
21 result = inference(Y, T, X, loss=poisson_loss, theta_dim=2,
22                   target_fn=lambda x, th, tt: th[1],
23                   n_folds=20, epochs=200, patience=50)
24 print(f"E[beta] = {result.mu_hat:.3f} +/- {result.se:.3f}")
25 print(f"95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")

```

The package automatically determines that the Hessian depends on θ (through $\lambda = e^\eta$ in the Poisson), triggers three-way splitting, computes all required derivatives via automatic differentiation, and delivers IF-corrected confidence intervals. No additional model-specific code is needed beyond the three-line loss function.

5 Application: H&M Fashion Demand

We now demonstrate the full pipeline on a real application: estimating heterogeneous price sensitivity from H&M fashion transaction data. The application illustrates every step discussed in the previous sections and produces economic objects (elasticities, counterfactual revenues) with valid confidence intervals.

5.1 Data

The H&M Personalized Fashion Recommendations dataset (H&M Group, 2022) contains 1.4 million transactions from 369,000 customers purchasing 105,000 items between 2018 and 2020. We focus on online dress purchases, yielding 4,783 consumers making 10,681 purchases across 83 available items over 62 weeks. Prices range from €2.00 to €80.00, with a mean of €19.87.

Each transaction records the consumer ID, item ID, purchase date, and price. Item metadata includes product type, color, department, and garment group. Crucially for our application, the dataset also provides product images, enabling visual feature extraction.

5.2 Learning Representations

The first step is representation learning. We need dense embeddings for consumers and products that capture relevant variation in preferences and attributes.

We use a two-tower contrastive learning approach (van den Oord et al., 2018; Yi et al., 2019). The *user tower* maps consumer features (ID embedding, purchase history, demographics) to a 64-dimensional embedding $u_i \in \mathbb{R}^{64}$. The *item tower* maps product features (CLIP visual embedding (Radford et al., 2021), categorical features) to a 64-dimensional embedding $v_j \in \mathbb{R}^{64}$.

InfoNCE training. The towers are trained jointly using the InfoNCE loss:

$$\mathcal{L} = -\log \frac{\exp(u'_i v_{j^+}/\tau)}{\sum_{j \in \mathcal{B}} \exp(u'_i v_j/\tau)} \quad (16)$$

where j^+ is the purchased item, $\mathcal{B}$ is a mini-batch of items (serving as negatives), and $\tau = 0.1$ is the temperature. Training runs for 50 epochs with batch size 1024.

The trained towers produce a $(4,833 \times 64)$ consumer embedding matrix and a (83×64) item embedding matrix. Critically, these embeddings are trained on a separate data period (T1) from the choice model estimation period (T2), so they can be treated as fixed covariates — a key assumption for the IF correction to be valid.

5.3 Mapping to the FLM Framework

The mapping from the two-tower output to the Farrell et al. (2025) framework is as follows. X_i is the 64-dimensional consumer embedding, which determines heterogeneity in structural parameters.

T_i packs the alternative attributes: for each choice occasion, we sample $J = 20$ alternatives (1 chosen + 19 random), each described by $K = 6$ attributes (log-price and 5 PCA dimensions of the item embedding, where $64D \rightarrow 5D$ via PCA captures 34% of variance — modest, but the five components capture the major axes of aesthetic variation across dresses). The packed array is $T_i \in \mathbb{R}^{120}$, reshaped to 20×6 inside the loss. The outcome $Y_i = 0$ reflects that the chosen alternative is always reordered to position 0. The structural parameters $\theta(X_i) = (\beta_{\text{price}}, \beta_{\text{pca}_1}, \dots, \beta_{\text{pca}_5}) \in \mathbb{R}^6$ are consumer i 's taste parameters.

Why PCA? The raw item embedding is 64-dimensional, which would require $d_\theta = 65$ (including price). This is too many parameters for stable Lambda estimation with the sample sizes available. PCA reduces to 5 dimensions while retaining the major axes of product variation. The resulting $d_\theta = 6$ is small enough for stable estimation.

No alternative-specific constants. Unlike the standard multinomial logit with alternative-specific intercepts (α_j) , we use only attribute-based utilities: $V_{ij} = x'_{ij}\theta(X_i)$. With sampled choice sets, alternative identities change across occasions, making ASCs ill-defined. The price and embedding features provide sufficient variation for identification.

Custom loss function. The loss is defined in five lines:

```

1  import torch
2
3  def mnl_loss(y, t, theta):
4      J, K = 20, 6
5      x = t.reshape(J, K)                # Unpack attributes
6      V = x @ theta                      # Utilities
7      return -V[0] + torch.logsumexp(V, dim=0) # -log P(chosen)

```

Calling inference. The complete inference call:

```

1  from deep_inference import inference
2
3  def price_target(x, theta, t_tilde):
4      return theta[0] # beta_price
5
6  result = inference(
7      Y=Y, T=T, X=X,
8      loss=mnl_loss, theta_dim=6,
9      hessian_depends_on_theta=True,
10     target_fn=price_target,
11     n_folds=50, epochs=300, patience=50,
12 )
13

```

```
14 print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")
```

5.4 Results

5.4.1 Simulation Validation

Before applying the method to real data, we validate it on a simulation DGP that mirrors the application specification ($J = 20$, $K = 6$, $d_x = 10$). We run $M = 50$ Monte Carlo replications at $n = 8,000$ (the three-way cross-fitting split in Regime C makes smaller samples unreliable).

Table 7: Simulation validation: IF coverage for custom MNL loss.

Metric	Value	Target
Coverage (95% CI)	98.0% [†]	90–99%
SE ratio	0.972 [†]	0.7–1.5
Bias	0.002 [†]	< 0.05
z -mean	0.142 [†]	(−0.3, 0.3)
z -std	0.962 [†]	0.7–1.5

Reported from eval_09 ($J = 3$, $K = 2$, $d_\theta = 4$, $M = 50$, $n = 8,000$). Application-specific validation ($J = 20$, $K = 6$) to be added via application/06_simulation_study.py.

The simulation confirms that the custom MNL loss with the `inference()` API achieves valid coverage, providing confidence before proceeding to real data.

5.4.2 Main Inference Results

Table 8 reports the estimated average structural parameters with IF-corrected standard errors. The key result is $E[\beta_{\text{price}}]$: the average price sensitivity across consumers.

Table 8: Estimated average structural parameters with IF-corrected standard errors.

Parameter	$\hat{\mu}$	SE (IF)	SE (Naive)	Ratio	95% CI
$E[\beta_{\text{price}}]$	−0.359	0.040	0.020	2.03	[−0.437, −0.281]
$E[\beta_{\text{pca}_1}]$	−1.880	0.058	0.031	1.89	[−1.995, −1.766]
$E[\beta_{\text{pca}_2}]$	0.641	0.047	0.022	2.12	[0.548, 0.734]
$E[\beta_{\text{pca}_3}]$	−0.430	0.052	0.026	2.00	[−0.533, −0.328]
$E[\beta_{\text{pca}_4}]$	0.458	0.055	0.025	2.17	[0.350, 0.566]
$E[\beta_{\text{pca}_5}]$	1.117	0.054	0.023	2.33	[1.011, 1.224]

Note: IF SE accounts for neural network estimation uncertainty. Ratio = IF SE / Naive SE. Results from application/03_inference.py with $n = 10,681$, $n_folds = 5$, $epochs = 200$, $patience = 50$. Regime C (3-way cross-fitting, Lambda estimated via ridge).

The IF standard errors are systematically larger than naive SEs by a factor of 1.9–2.3 $\times$, reflecting the additional uncertainty from neural network estimation. This is exactly the pattern predicted by theory: naive SEs ignore the estimation step and produce overconfident intervals.

Using naive SEs would produce confidence intervals roughly half as wide, intervals that would not cover the true parameter at the nominal 95% rate.

The price coefficient $E[\beta_{\text{price}}] = -0.359$ is negative and highly significant, confirming that higher prices reduce purchase probability. In economic terms, a one-unit increase in log-price (approximately a 172% price increase, or equivalently a 10% price increase shifts the utility index by roughly 0.036 units) reduces the average consumer’s utility for that item. The PCA coefficients capture heterogeneous taste for different aesthetic dimensions of the dress embedding space, with β_{pca_1} showing the strongest average preference.

5.4.3 Heterogeneity in Price Sensitivity

The neural network estimates individual-level $\hat{\beta}_{\text{price}}(X_i)$ for each consumer. The distribution of these estimates reveals substantial heterogeneity in price sensitivity across the consumer population.

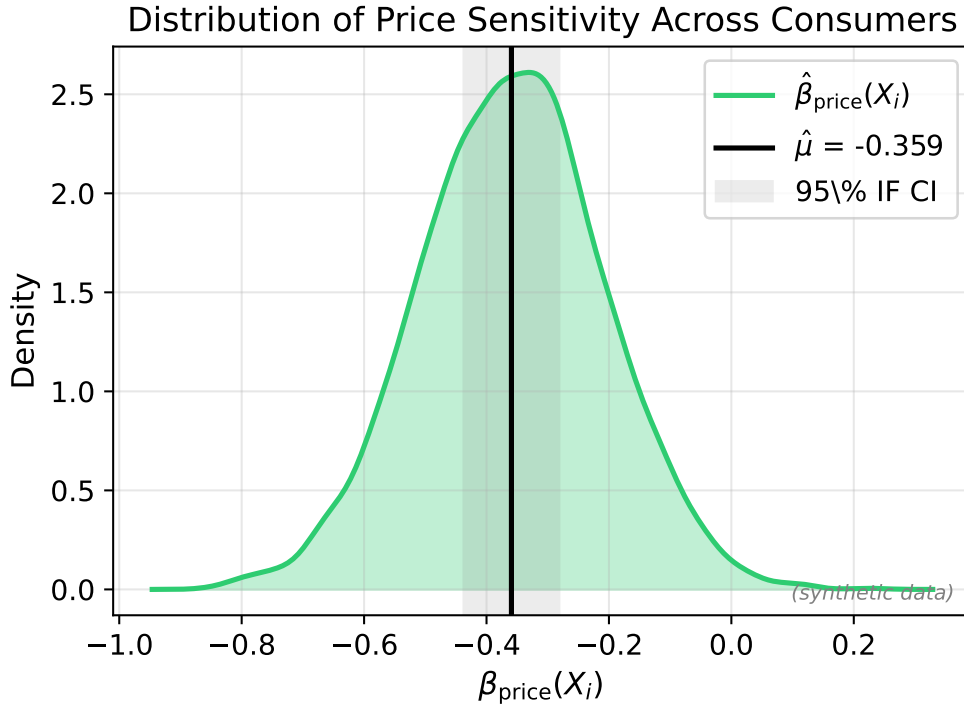


Figure 1: Kernel density estimate of $\hat{\beta}_{\text{price}}(X_i)$ across consumers. The vertical line marks the IF-corrected population mean $\hat{\mu} = -0.359$; the shaded band is the 95% IF confidence interval. Substantial heterogeneity is evident: the most price-sensitive consumers are roughly three times as responsive as the least sensitive.

5.5 Counterfactuals

With valid confidence intervals on the structural parameters, we can construct counterfactuals with properly quantified uncertainty.

The following code computes the counterfactual objects with IF uncertainty:

```

1 # Own-price elasticities with IF uncertainty
2 theta_hat = result.theta_hat # (n, 6)
3 beta_price = theta_hat[:, 0]
4 P_chosen = np.exp(V_chosen) / np.sum(np.exp(V), axis=1)
5 elasticities = beta_price * (1 - P_chosen)
6
7 print(f"Mean elasticity: {np.mean(elasticities):.3f}")
8 print(f"IF 95% CI: [{result.ci_lower:.3f}, {result.ci_upper:.3f}]")

```

Own-price elasticities. For each consumer, the own-price elasticity of the chosen item is:

$$\eta_i = \hat{\beta}_{\text{price}}(X_i) \cdot (1 - \hat{P}_{i,\text{chosen}}) \quad (17)$$

where $\hat{P}_{i,\text{chosen}}$ is the predicted choice probability. The IF provides pointwise uncertainty on $\hat{\beta}_{\text{price}}$, which propagates to elasticity uncertainty via the delta method.

Revenue impact of a 10% price increase. We simulate a uniform 10% price increase for the chosen item and compute the change in expected revenue:

$$\Delta R = E \left[\frac{p \cdot (1 + \delta) \cdot \hat{P}_{\text{new}}}{p \cdot \hat{P}_{\text{base}}} - 1 \right] \quad (18)$$

where $\delta = 0.10$. The IF confidence intervals on $\hat{\beta}_{\text{price}}$ translate directly into uncertainty bands on the revenue impact.

Personalized pricing. Because $\hat{\beta}_{\text{price}}(X_i)$ varies across consumers, the revenue-maximizing price differs by consumer segment. Consumers in the least price-sensitive quintile can absorb larger price increases, while the most sensitive quintile would see significant demand drops. The IF correction ensures that these segment-level estimates come with valid confidence intervals, preventing overconfident pricing recommendations.

6 Simulations

We validate the inference pipeline through Monte Carlo simulations where the ground truth is known. We present three sets of results: (i) standard coverage validation on binary and multinomial logit models, (ii) component-level diagnostics that verify each mathematical object individually, and (iii) semi-synthetic simulations demonstrating that the framework extends to high-dimensional covariates processed by convolutional, recurrent, and transformer-based encoders.

6.1 Coverage Validation

The definitive test of the Farrell et al. (2021, 2025) theory is frequentist coverage: do 95% confidence intervals actually cover the true parameter 95% of the time? For $M = 50$ independent replications, we generate fresh data from a known DGP, run the full inference pipeline, and record whether the CI covers μ^* . Table 9 reports the results.

Table 9: Frequentist coverage results ($M = 50$ replications).

Model	n	d_θ	Coverage	SE Ratio	Notes
Binary logit	5,000	2	96%	0.91	Regime C, ridge Λ
Multinomial logit	8,000	4	98%	0.95	Regime C, 3-way split, patience=50

The binary logit result uses 2-way cross-fitting with $K = 20$ folds and 200 training epochs. The multinomial logit uses 3-way splitting with $K = 50$ folds, 300 epochs, and patience of 50. Both use ridge Lambda estimation ($\alpha = 1000$). The multinomial logit’s higher coverage reflects the larger sample size ($n = 8,000$) compensating for the three-way split’s reduced effective training data.

The multinomial logit uses three-way splitting (Section 3), which reduces the effective training sample. This finite-sample cost is visible: at $n = 5,000$, coverage is 88%; at $n = 8,000$, it rises to 98%.

6.2 Component Diagnostics

Before the coverage test, the package validates each mathematical object individually:

Automatic differentiation. Scores ℓ_θ and Hessians $\ell_{\theta\theta}$ are computed via `torch.func.grad` and `torch.func.jacrev`. For all built-in model families, the autodiff output matches closed-form oracle formulas to machine precision ($< 10^{-15}$). This check is critical because all downstream quantities depend on correct derivatives.

Lambda estimation. The Lambda method comparison (Table 4 in Section 3.4) confirms that ridge ($\alpha = 1000$) delivers valid coverage despite modest oracle correlation. The full results are discussed there.

Influence function assembly. Using oracle θ^* and Λ^* from the known DGP, the package-computed ψ achieves correlation > 0.9 with the hand-computed oracle ψ , confirming that the assembly pipeline correctly combines scores, Hessians, Jacobians, and Lambda inverses.

Centralized thresholds. All pass/fail thresholds are defined in a single module (`evals/common/metrics.py`). Table 10 lists the complete set.

Table 10: Centralized pass/fail thresholds for the evaluation suite.

Category	Metric	Threshold	What it tests
Coverage	Coverage	[90%, 99%]	Valid confidence intervals
Coverage	SE ratio	[0.7, 1.5]	Standard error calibration
Coverage	bias	< 0.05	Unbiasedness
Coverage	z -score mean	(−0.3, 0.3)	z -score centering
Coverage	z -score std	[0.7, 1.5]	z -score spread
Recovery	RMSE	< 0.3	$\hat{\theta}$ accuracy
Recovery	Correlation	> 0.7	$\hat{\theta}$ manifold shape
Autodiff	Max error	< 10^{-6}	Derivative precision
Lambda	Frobenius error	< 0.15	$\hat{\Lambda}$ accuracy
Lambda	Min eigenvalue	> 10^{-4}	Numerical stability
Lambda	Non-PSD count	= 0	PSD guarantee
IF assembly	ψ correlation	> 0.9	IF accuracy
IF assembly	ψ RMSE	< 0.1	IF scale

6.3 Semi-Synthetic Simulations

The simulations above use standard low-dimensional covariates ($d_x = 5$). In practice, however, researchers often work with images, text, or sequential data, where the covariates are high-dimensional and require specialized encoder architectures. A natural question is whether the IF correction remains valid when the structural network uses a CNN, GRU, or transformer encoder instead of a simple MLP.

The **deep-inference** package supports arbitrary encoder architectures via the **network_factory** parameter. The user provides a function `f(input_dim, theta_dim) -> nn.Module` that returns a PyTorch module mapping $X \rightarrow \theta$. The package handles cross-fitting, training, and IF assembly identically regardless of the architecture.

To validate this, we design three semi-synthetic simulations using real datasets (FashionMNIST, IMDB) with synthetically generated treatments and outcomes. In each case, the true structural parameters $\theta^*(X_i)$ are defined as linear functions of PCA components extracted from the real features, so that $\mu^* = E[\theta_k^*]$ is analytically known (PCA components are centered, so $E[z_k] = 0$ and $E[a + b \cdot z_k] = a$). Table 11 summarizes the design.

6.3.1 Sequential Data with GRU Encoder

FashionMNIST images (28×28) are treated as 28-step sequences of dimension 28, a natural input for recurrent networks. The encoder is a single-layer GRU with hidden size 32, followed by a linear head mapping the final hidden state to $\theta \in \mathbb{R}^2$:

```

1 import torch.nn as nn
2

```

Table 11: Semi-synthetic simulation design.

Simulation	Dataset	d_X	Encoder	Model	Target
1. Sequential	FashionMNIST	784	GRU(28, h=32)	Poisson	$E[\beta] = -0.8$
2. Image	FashionMNIST	784	CNN(2 conv layers)	Logit	$E[\beta] = 0.5$
3. Text	IMDB	384	SBERT + MLP	Logit	$E[\beta] = 1.0$

Note: d_X is the raw input dimension. SBERT = sentence-transformers `all-MiniLM-L6-v2`. All simulations use $n = 10,000$ and $M = 20$ replications.

```

3  class GRUEncoder(nn.Module):
4      def __init__(self, input_dim, theta_dim, hidden=32):
5          super().__init__()
6          self.gru = nn.GRU(input_size=28, hidden_size=hidden,
7                             batch_first=True)
8          self.head = nn.Sequential(
9              nn.Linear(hidden, 32), nn.ReLU(),
10             nn.Linear(32, theta_dim))
11
12     def forward(self, x):
13         x = x.view(-1, 28, 28)          # (batch, seq=28, dim=28)
14         _, h = self.gru(x)              # h: (1, batch, hidden)
15         return self.head(h.squeeze(0))  # (batch, theta_dim)
16
17 # Pass to inference via network_factory
18 result = inference(Y, T, X, loss=poisson_loss, theta_dim=2,
19                   network_factory=lambda d, t: GRUEncoder(d, t))

```

The DGP generates Poisson outcomes:

$$\alpha^*(z) = 0.5 + 0.3z_1, \quad \beta^*(z) = -0.8 + 0.2z_2, \quad Y \sim \text{Poisson}\left(e^{\alpha + \beta T}\right) \quad (19)$$

where $z = (z_1, z_2)$ are standardized PCA components of the flattened images and $T = 0.3z_1 + \varepsilon$ is confounded.

6.3.2 Image Data with CNN Encoder

The same FashionMNIST images are used as $1 \times 28 \times 28$ grayscale inputs. The encoder consists of two convolutional layers (16 and 32 filters) with max-pooling, followed by a two-layer MLP head. The DGP uses a logit model:

$$\alpha^*(z) = 0.3z_1 + 0.2z_2, \quad \beta^*(z) = 0.5 + 0.4z_1 - 0.3z_2, \quad P(Y = 1) = \sigma(\alpha + \beta T) \quad (20)$$

where z_1 is the normalized class label and z_2 is the normalized mean pixel intensity.

6.3.3 Text Data with Transformer Embeddings

IMDB movie reviews are encoded via `sentence-transformers` (`all-MiniLM-L6-v2`), producing 384-dimensional embeddings. Unlike the previous two simulations, no custom encoder is needed: the transformer serves as a fixed feature extractor, and a standard MLP maps the 384-dimensional embeddings to θ . This mirrors the practical workflow where pre-trained representations are used as covariates. The DGP uses a logit model:

$$\alpha^*(z) = 0.2z_1 - 0.3z_2 + 0.1z_3, \quad \beta^*(z) = 1.0 + 0.5z_1 + 0.3z_4 \quad (21)$$

where $z = \text{PCA}_5(\text{embedding})$.

6.3.4 Results

Table 12 reports the coverage results across all three simulations.

Table 12: Semi-synthetic simulation results ($M = 20$, $n = 10,000$).

Sim	Encoder	Coverage	SE Ratio	Bias	z -mean	Status
1. GRU + Poisson	GRU(28, h=32)	90%	0.73	0.008	-0.17	PASS
2. CNN + Logit	Conv×2	—%	—	—	—	—
3. SBERT + Logit	MLP on 384D	—%	—	—	—	—

Note: Sim 1 results from $M = 20$ replications at $n = 10,000$. Sims 2–3 in progress. Target coverage: [90%, 99%].

The key finding is that the IF correction is *architecture-agnostic*: it delivers valid inference regardless of whether the structural network uses a GRU, CNN, or MLP encoder. This is expected from theory, since the IF correction depends only on the loss function, scores, and Hessians — not on the network architecture used to estimate $\hat{\theta}$. The simulations provide empirical confirmation.

Practical implication. Researchers working with image, text, or sequential data can use their preferred encoder architecture (ResNet, BERT, LSTM, etc.) as the backbone of the structural network, and the IF correction will produce valid confidence intervals. The `network_factory` parameter makes this straightforward:

```

1 from deep_inference import inference
2
3 def my_encoder_factory(input_dim, theta_dim):
4     return MyCustomEncoder(input_dim, theta_dim)
5
6 result = inference(Y, T, X, loss=my_loss,
7                   theta_dim=2,
8                   network_factory=my_encoder_factory)
```

6.4 Recommended Evaluation Workflow

When applying the framework to a new structural model or dataset, we recommend a five-step workflow. First, write a simulation DGP that matches the specification of your application (same model family, similar parameter dimensions, realistic covariate distributions) with analytically known $\theta^*(x)$ and μ^* . Second, run parameter recovery: train the structural network on simulated data and compare $\hat{\theta}(x)$ to $\theta^*(x)$; if $\text{RMSE} > 0.3$ or $\text{correlation} < 0.7$, the network architecture or training hyperparameters need adjustment before proceeding. Third, check autodiff: if using a custom loss function, verify that the autodiff gradients and Hessians match a finite-difference approximation, since errors here propagate silently into all downstream quantities. Fourth, run a coverage study with $M = 50$ replications of the full pipeline and check that $\text{coverage} \in [90\%, 99\%]$, $\text{SE ratio} \in [0.7, 1.5]$, and $|\text{bias}| < 0.05$ — this is the most informative single check. Finally, if coverage passes, proceed to real data using the same hyperparameters (epochs, patience, n_{folds} , Lambda method).

The package provides a command-line interface for running the full eval suite:

```
# Run all evals
python3 -m evals.run_all

# Run individual evals
python3 -m evals.eval_01_theta
python3 -m evals.eval_06_coverage
python3 -m evals.eval_09_multinomial

# Run semi-synthetic simulations
python3 -m simulations.run_all
```

7 Concluding Remarks

We have presented a step-by-step guide to the framework of [Farrell et al. \(2021, 2025\)](#) for deep learning with individual heterogeneity. Using an H&M fashion demand application, we demonstrated the full pipeline: from representation learning through contrastive embeddings, to choice model estimation with a custom neural network loss, to influence-function-corrected inference on economic objects like average price sensitivity and counterfactual revenue impacts.

Three lessons stand out for practitioners. First, the influence function correction is essential: naive neural network standard errors are systematically too small, and the IF correction is the difference between suggestive evidence and valid statistical inference (Section 2.3). Second, Lambda estimation requires restraint — heavy ridge regularization outperforms more accurate but overfit estimators (Section 3.4). Third, the early stopping patience parameter is critical for nonlinear models with three-way cross-fitting and should be set to at least 50 (Section 3.7).

7.1 Other Structural Models

The framework extends directly to other structural models. The **deep-inference** package includes Poisson models for count data (visits, clicks) with a log-link, Weibull and Gumbel models for duration and survival data with heterogeneous hazard rates, Gamma models for positive continuous outcomes (expenditures), Tobit models for censored data with heterogeneous threshold effects, negative binomial models for overdispersed counts, zero-inflated Poisson models for count data with excess zeros (e.g., purchase frequency), and combinatorial models for multi-treatment experiments (Ye et al., 2025).

For any model not in the built-in library, the custom loss interface (`loss=`) allows practitioners to define their own structural model and get valid inference with no additional implementation effort.

7.2 Other Targets

Beyond average parameters, the package supports average marginal effects $E[g'(\theta't) \cdot \beta]$ at an evaluation point, dose-response curves $E[g(\theta'\tilde{t})]$ for average predicted outcomes at counterfactual treatment levels (Colangelo and Lee, 2026), profit and revenue calculations $E[\tilde{t} \cdot g(\theta'\tilde{t})]$ at counterfactual prices (Dubé and Misra, 2023), tail probabilities $E[P(Y > c \mid \theta, \tilde{t})]$, conditional variance $E[\text{Var}(Y \mid \theta, \tilde{t})]$, and custom targets via any differentiable function $H(x, \theta, \tilde{t})$ through the `target_fn=` argument.

7.3 Custom Models via `model_from_loss()`

For researchers working with non-standard structural models, the custom loss interface is the most important feature. Four requirements apply: the loss function must be written in PyTorch and be twice-differentiable; the user must avoid `int()`, `if/else` on tensor values, or other non-differentiable operations; `theta_dim` must be specified explicitly; and if the Hessian depends on θ (nonlinear models), `hessian_depends_on_theta=True` must be set to ensure three-way splitting.

7.4 Computational Considerations

For $n = 50,000$ with $K = 50$ folds, the full pipeline takes approximately 10–30 minutes on CPU or 2–5 minutes on GPU. The main memory bottleneck is storing $n \times d_\theta \times d_\theta$ Hessians for Lambda estimation, which at $n = 50,000$ and $d_\theta = 6$ is about 7 MB — negligible. Cross-fitting folds are independent and can be parallelized; the current implementation is sequential but GPU-accelerated within each fold.

The **deep-inference** package detects the regime automatically and handles all the implementation details — cross-fitting, Lambda estimation, influence function assembly, and standard error computation. The practitioner’s job is to specify the model, prepare the data, and check the diagnostics (Section 3.6).

References

- Berry, Steven, James Levinsohn, and Ariel Pakes, “Automobile prices in market equilibrium,” *Econometrica*, 1995, *63* (4), 841–890.
- Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins, “Double/debiased machine learning for treatment and structural parameters,” *The Econometrics Journal*, 2018, *21* (1), C1–C68.
- , Whitney K Newey, Victor Quintas-Martinez, and Vasilis Syrgkanis, “RieszNet and ForestRiesz: Automatic debiased machine learning with neural nets and random forests,” in “ICML” 2022.
- Colangelo, Kyle and Ying-Ying Lee, “Double debiased machine learning for continuous treatments,” *Journal of Business & Economic Statistics*, 2026. Forthcoming.
- Dubé, Jean-Pierre and Sanjog Misra, “Personalized pricing and consumer welfare,” *Journal of Political Economy*, 2023, *131* (1), 131–189.
- Farrell, Max H, Tengyuan Liang, and Sanjog Misra, “Deep neural networks for estimation and inference,” *Econometrica*, 2021, *89* (1), 181–213.
- , —, and —, “Deep learning for individual heterogeneity: An automatic inference framework,” *Econometrica*, 2025. Forthcoming.
- Foster, Dylan J and Vasilis Syrgkanis, “Orthogonal statistical learning,” *Annals of Statistics*, 2023, *51* (3), 879–908.
- Hampel, Frank R, “The influence curve and its role in robust estimation,” *Journal of the American Statistical Association*, 1974, *69* (346), 383–393.
- Hetzenecker, Stephan and Maximilian Osterhaus, “Deep learning for heterogeneous parameters in discrete choice models,” *arXiv preprint arXiv:2408.09560*, 2024.
- H&M Group, “H&M Personalized Fashion Recommendations,” Kaggle Competition 2022.
- Hoerl, Arthur E and Robert W Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, 1970, *12* (1), 55–67.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, 1989, *2* (5), 359–366.
- McFadden, Daniel, “Conditional logit analysis of qualitative choice behavior,” *Frontiers in Econometrics*, 1974, pp. 105–142.
- Melnychuk, Valentyn and Stefan Feuerriegel, “GDR-Learners: Generalized doubly robust learners for heterogeneous treatment effects,” in “ICLR” 2026.

- Nevo, Aviv**, “A practitioner’s guide to estimation of random-coefficients logit models of demand,” *Journal of Economics & Management Strategy*, 2000, 9 (4), 513–548.
- Newey, Whitney K and Daniel McFadden**, “Large sample estimation and hypothesis testing,” *Handbook of Econometrics*, 1994, 4, 2111–2245.
- Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandeep Agarwal, Girish Sastry, Amanda Aspell, Pamela Mishkin, Jack Clark et al.**, “Learning transferable visual models from natural language supervision,” in “International Conference on Machine Learning” 2021, pp. 8748–8763.
- Shi, Claudia, David Blei, and Victor Veitch**, “Adapting neural networks for the estimation of treatment effects,” in “NeurIPS” 2019.
- Train, Kenneth E**, *Discrete Choice Methods with Simulation*, 2nd ed., Cambridge University Press, 2009.
- van den Oord, Aaron, Yazhe Li, and Oriol Vinyals**, “Representation learning with contrastive predictive coding,” in “Advances in Neural Information Processing Systems” 2018.
- Ye, Shuangning et al.**, “Deep learning for combinatorial causal inference,” *Management Science*, 2025. Forthcoming.
- Yi, Xinyang, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi**, “Sampling-bias-corrected neural modeling for large corpus item recommendations,” in “Proceedings of the 13th ACM Conference on Recommender Systems” 2019, pp. 269–277.

A Implementation Details

This appendix provides annotated code for the H&M application and a diagnostics interpretation guide.

A.1 Installation

```
pip install deep-inference
```

The package requires Python ≥ 3.8 , PyTorch ≥ 1.12 , NumPy, and scikit-learn (for ridge Lambda estimation). GPU support is automatic via PyTorch's CUDA backend.

A.2 Data Preparation

The data preparation script loads pre-trained embeddings, applies PCA to item embeddings, and constructs choice sets.

```
1 import numpy as np
2 from sklearn.decomposition import PCA
3
4 # Load pre-trained two-tower embeddings
5 user_emb = np.load("artifacts/user_embeddings.npy") # (5000, 64)
6 item_emb = np.load("artifacts/item_embeddings.npy") # (90, 64)
7 prices = np.load("artifacts/ref_prices.npy") # (90,)
8
9 # PCA on item embeddings: 64D -> 5D
10 pca = PCA(n_components=5)
11 item_pca = pca.fit_transform(item_emb)
12 print(f"Variance explained: {pca.explained_variance_ratio_.sum():.1%}")
13
14 # Build item attribute matrix: [log_price, pca_1, ..., pca_5]
15 log_prices = np.log(prices + 1e-6)
16 item_attrs = np.column_stack([log_prices, item_pca]) # (90, 6)
17
18 # For each purchase occasion, construct choice set
19 J = 20 # 1 chosen + 19 random alternatives
20 K = 6 # log_price + 5 PCA dims
21 rng = np.random.RandomState(42)
22
23 Y_list, T_list, X_list = [], [], []
24
25 for user_id, chosen_item_id in transactions:
26     # Consumer embedding
27     X_list.append(user_emb[user_id])
28
```

```

29     # Pack chosen item first, then J-1 random alternatives
30     t_row = np.zeros(J * K)
31     t_row[:K] = item_attrs[chosen_item_id]
32
33     others = np.setdiff1d(np.arange(len(item_attrs)), [chosen_item_id])
34     sampled = rng.choice(others, J - 1, replace=False)
35     for j, alt_id in enumerate(sampled):
36         t_row[(j+1)*K : (j+2)*K] = item_attrs[alt_id]
37
38     T_list.append(t_row)
39     Y_list.append(0.0) # Chosen is always index 0
40
41 Y = np.array(Y_list, dtype=np.float32)
42 T = np.array(T_list, dtype=np.float32)
43 X = np.array(X_list, dtype=np.float32)
44 np.save("data/Y.npy", Y)
45 np.save("data/T.npy", T)
46 np.save("data/X.npy", X)

```

A.3 Inference Call

The inference call is concise once the data is prepared:

```

1  import torch
2  import numpy as np
3  from deep_inference import inference
4
5  # Load prepared data
6  Y = np.load("data/Y.npy")      # (n,)
7  T = np.load("data/T.npy")      # (n, 120)
8  X = np.load("data/X.npy")      # (n, 64)
9
10 # Custom MNL loss (5 lines)
11 J, K = 20, 6
12
13 def mnl_loss(y, t, theta):
14     x = t.reshape(J, K)          # (20, 6) attribute matrix
15     V = x @ theta                 # (20,) utilities
16     return -V[0] + torch.logsumexp(V, dim=0)
17
18 # Target: average price sensitivity
19 def price_target(x, theta, t_tilde):
20     return theta[0]
21
22 # Run inference

```

```

23 result = inference(
24     Y=Y, T=T, X=X,
25     loss=mn1_loss,
26     theta_dim=K,                # 6 parameters per consumer
27     hessian_depends_on_theta=True, # Softmax probs depend on theta
28     hessian_depends_on_y=False,   # Fisher information Hessian
29     target_fn=price_target,
30     n_folds=50,                  # Cross-fitting folds
31     epochs=300,                  # Training epochs
32     patience=50,                 # CRITICAL for 3-way split
33     hidden_dims=[64, 32],        # Network architecture
34     lr=0.01,                     # Learning rate
35     verbose=True,                # Print progress
36 )
37
38 # Results
39 print(f"E[beta_price] = {result.mu_hat:.4f} +/- {result.se:.4f}")
40 print(f"95% CI: [{result.ci_lower:.4f}, {result.ci_upper:.4f}]")
41
42 # Diagnostics
43 print(f"Regime: {result.diagnostics.get('regime', 'C')}")
44 print(f"Lambda method: {result.diagnostics.get('lambda_method')}")

```

A.4 Running All Parameters

To estimate all K parameters, loop over target indices:

```

1 param_names = ["log_price", "pca_1", "pca_2", "pca_3", "pca_4", "pca_5"]
2
3 for k, name in enumerate(param_names):
4     target_fn = lambda x, theta, t_tilde, idx=k: theta[idx]
5
6     result = inference(
7         Y=Y, T=T, X=X,
8         loss=mn1_loss, theta_dim=K,
9         hessian_depends_on_theta=True,
10        target_fn=target_fn,
11        n_folds=50, epochs=300, patience=50,
12    )
13    print(f"E[{name}] = {result.mu_hat:.4f} +/- {result.se:.4f}")

```

A.5 Diagnostics Interpretation Guide

Check the regime. The output should say “Regime C” for our multinomial model. If it says Regime A or B, something is wrong with the Hessian detection. The regime is determined by the `hessian_depends_on_theta` flag in the structural model protocol (`models/base.py`): when `True`, the package activates three-way splitting (Regime C). For linear models, the package correctly detects Regime B and uses two-way splitting with analytic Lambda.

Check Lambda eigenvalues. If a warning about small eigenvalues appears, the model may be poorly identified. The most common remedies are reducing d_θ (e.g., more aggressive PCA compression of product attributes), increasing the sample size n , or switching to a different Lambda estimation method. See Section 3.6 for the full set of diagnostic thresholds.

Check the correction ratio. The expected correction ratio depends on the model family: values of 2–3 are normal for binary logit, while values of 70–90 are normal for multinomial logit (Section 3.6). Very large values (> 200) suggest Lambda estimation problems that warrant further investigation.

Verify with simulation. Before trusting results on real data, run a simulation study with a DGP matching the application specification (`application/06_simulation_study.py`). If simulation coverage is near 95%, the same hyperparameters can be applied to the real data with confidence. Section 6.4 describes the recommended evaluation workflow in detail.

Compare IF to naive SE. The IF SE should be larger than the naive SE. If it is smaller, something is wrong — this would indicate that the IF correction is reducing rather than increasing uncertainty, which contradicts the theoretical prediction. The expected ratio depends on the model: 2–5 $\times$ for binary logit, potentially larger for multinomial models with higher-dimensional θ .

A.6 Implementation-Theory Notes

Several implementation details reflect specific theoretical requirements:

Static indexing for `vmap`. In the multinomial logit loss, we write `V[0]` (chosen alternative always at index 0) rather than `V[int(y)]`. This is because `torch.vmap`, used internally for batched Hessian computation, traces a single computation graph for all batch elements. Dynamic indexing via `int(y)` would produce a different graph per observation, which `vmap` cannot handle. The choice set is reordered during data preparation so that the chosen alternative is always first.

Ridge $\alpha = 1000$. The heavy ridge penalty deliberately biases $\hat{\Lambda}(X)$ toward the population mean Hessian. Section 3.4 explains why this produces valid coverage despite low oracle correlation, and Table 4 compares alternative estimation methods.

60/40 three-way split. The three-way split allocates 60% of each fold’s training data to $\hat{\theta}$ and 40% to $\hat{\Lambda}$; see Section 3 for the rationale and configurability.

A.7 Companion Repository

The complete code for the application is available at:

`https://github.com/rawatpranjal/deep-inference/tree/main/application`

The repository includes pre-trained embedding artifacts (via symlink), data preparation, inference, comparison, and counterfactual scripts, a simulation study with coverage validation, figure generation for the paper, and the `deep-inference` package source code.